

An Efficient Parallel Scheme for Run-Time Scheduling

Tsung-Chuan Huang¹ and Po-Hsueh Hsu²

¹Department of Electrical Engineering, National Sun Yat-sen University
tch@mail.nsysu.edu.tw

²Department of Electronic Engineering, Cheng Shiu Institute of Technology
phhsu@cc.csit.edu.tw

Abstract

High performance computing capability is crucial for the advanced calculations of scientific applications. A parallelizing compiler can take a sequential program as input and automatically translate it into a parallel form. But for loops with arrays of irregular (i.e., indirectly indexed), nonlinear or dynamic access patterns, no state-of-the-art compilers can determine their parallelism at compile-time. In this paper, we propose an efficient parallel scheme to compute a high parallelism execution schedule for those loops. This new scheme first constructs a predecessor iteration table in inspector phase, and then schedules the whole loop iterations into wavefronts for parallel execution. For non-uniform access patterns, the performance of the inspector/executor methods usually degrades dramatically, but it is not valid for our scheme. Furthermore, this scheme is especially suitable for multiprocessor systems because of the features of high scalability and low overhead.

1 Introduction

Recently, automatic parallelization is a key enabling technique for parallel computing. How to exploit the parallelism in a loop, or the *loop parallelization*, is an important issue in this area. Current parallelizing compilers demonstrate their effectiveness for loops that have no cross-iteration dependences or have only uniform dependences. But there are some limitations in the parallelization of

loops with complex or statically insufficiently defined access patterns.

In order to convert sequential programs to parallel equivalents, parallelizing compilers must perform data dependence analysis first to determine whether a loop, or part of it, can be executed in parallel without violating the original semantics. This analysis is mainly focused on array subscript expressions, i.e., array access patterns. Basically, the application programs can be classified into two types:

1. Regular programs, in which memory accesses are described by linear equations of variables (usually loop index variables).
2. Irregular programs, in which memory accesses are described by indirection mapping (e.g., index arrays) or computation dependent.

Regular programs are much easier to deal with because they can be analyzed at compile-time. Unfortunately, many scientific programs performing complex modeling or simulations, such as DYNA-3D and SPICE, are usually irregular programs. The form of irregular accesses looks like $A(w(i))$ or $A(idx)$, where w is an index array and idx is computed inside the loop but not an induction variable. In the circumstances, we can only resort to run-time parallelization techniques as complementary solutions because information for analysis is not available until program execution.

2 Related Work

Two different approaches have been developed for run-time loop parallelization: the speculative doall execution and the inspector/executor parallelization method. The former approach assumes the loop to be fully parallelizable and executes it speculatively, then examines the correctness of parallel execution after loop termination. In the latter approach, the inspector examines cross-iteration dependences and produces a parallel execution schedule first, and then the executor performs actual loop operations based on the schedule arranged by the inspector.

Speculative Doall Parallelization. The speculative doall parallelization speculatively executes the loop operations in parallel, accompanied with a marking mechanism to track the accesses to the target arrays. After loop termination, an analysis mechanism is applied to examine whether this speculative doall parallelization passes or not, i.e., to check whether no cross-iteration dependence occurs in the loop. If it passes, a significant speedup will be obtained. Otherwise, the altered variables should be restored and the loop is re-executed serially. The reader who is interested in this topic may refer to Huang and Hsu [2], and Rauchwerger and Padua [9].

Run-Time Doacross Parallelization (The Inspector/Executor Method). According to the scheduling unit, we classify the inspector/executor methods into two types: the reference-level and the iteration-level. The reference-level type assumes a memory reference of the loop body as the basic unit of scheduling and synchronization in the inspector. Busy-waits are used to ensure values are produced before used during the executor phase. This type of method has the advantage of increasing the overlap of dependent iterations, but at the expense of more synchronization overhead. The reader who is interested in this topic may see Chen et al. [1], and Xu and Chaudhary [11].

The iteration-level type assumes loop iteration as the basic scheduling unit in the inspector. The inspector schedules

the source loop iterations into appropriate wavefronts at run-time; wavefronts will be executed serially but the iterations in a wavefront are executed in parallel. The executor then performs actual execution according to the wavefront sequence. The reader who is interested in this topic may see Zhu and Yew [12], Midkiff and Padua [6], Polychronopoulos [7], Saltz *et al.* [10], Leung and Zahorjan [4], Leung and Zahorjan [5], Rauchwerger *et al.* [8], and Huang *et al.* [3].

In general, speculative doall execution gains significant speedup if the target loop is intrinsically fully parallel; otherwise a hazard arises when cross-iteration dependences occur [2, 9]. In contrast, inspector/executor methods are profitable in extracting doacross loop parallelism, but may suffer a relative amount of processing overhead and synchronization burden [1, 3-8, 10-12].

3 Our Efficient Parallel Scheme

In retrospect of the development of run-time doacross parallelization, we find some inefficient factors as follows:

- Sequential inspector,
- Synchronization overhead on updating shared variables,
- Overhead in constructing dependence chains for all array elements,
- Large memory space in operation,
- Inefficient scheduler,
- Possible load migration from inspector to executor.

In this paper, we propose an efficient parallel scheme to overcome these problems. Our new scheme constructs an immediate predecessor table first, and then schedules the whole loop iterations efficiently into wavefronts for parallel execution. Owing to the characteristics of high scalability and low overhead, our scheme is especially suitable for multiprocessor systems.

3.1 Design Considerations

As described in the previous section, the benefits coming from run-time parallelization may be offset by a relative amount of processing overhead. Consequently, how to reduce the processing overhead is a major concern of our method. Designing a parallel inspector and parallel inspector to exploit the capability of multiprocessing, herein, becomes the objective that we pursue.

Rauchwerger *et al.* [8] designed a run-time parallelization method, which is fully parallel, with no synchronization, and can be applied on any kind of loop. This scheme is based on the operation of predecessor references; we call it *PRS* (Predecessor_Reference_Scheme) for short. Their inspector encodes the predecessor/successor information, for the references to $A(x)$, in a reference array R_x and a hierarchy vector H_x so that the scheduler can arrange loop iterations according to this dependence information. The scheduler is easy to implement but not efficient enough. In order to identify the iterations belonging to the i th wavefront, *all the references* must be examined to determine the ready states of corresponding unscheduled iterations in the i th step. Thus, the scheduler takes $O((numref/numproc) * cpl)$ time for this processing, where $numref$ is the number of references, $numproc$ is the number of processors, and cpl is the length of the critical path in the directed acyclic graph that describes the cross-iteration dependency in the loop. In fact, the above processing overhead can be completely eliminated if we use the data representation of *predecessor iteration* instead of *predecessor reference*.

Hereby, we develop a new run-time scheme to automatically extract the parallelism of doacross loops under the following considerations:

1. The inspector is parallel and efficient to construct a predecessor iteration table for recording the information of iteration dependence.
2. The scheduler is parallel and efficient to produce the wavefront

schedule with the help of predecessor iteration table.

3. The scheme should be no synchronization to ensure good scalability.

3.2 An Example

We demonstrate the operation of our run-time scheme by an example. Fig. 1(a) is an irregular loop to be parallelized. The access pattern of the loop is shown in Fig. 1(b). In our scheme, a predecessor iteration table, with the help of an auxiliary array la (which will be explained in the next subsection), is constructed first as shown in Fig. 1(c) at run-time. Then, the wavefronts of loop iterations can be quickly scheduled out there in Fig. 1(d). The predecessor iteration table is implemented by two arrays, $pw(1:numiter)$ and $pr(1:numiter)$, where $numiter$ is the number of iterations of the target loop. Element $pw(i)$ records the predecessor iteration of iteration i that accesses (either *read* or *write*) the same array element $x(w(i))$. A similar explanation applies to $pr(i)$. After the predecessor iteration table is built, the scheduler uses this information to arrange iterations into wavefronts: firstly, the iterations with no predecessor iteration (i.e. whose associated elements in pw and pr are both zero) are scheduled into the first wavefront, then the iterations whose predecessor iterations have been scheduled are assigned into the next wavefront. The procedure is repeated until all the iterations are scheduled.

3.3 The Parallel Inspector

The goal of our inspector is to construct a predecessor iteration table *in parallel*. The iterations of target loop are distributed in block faction into processors; each processor takes charge of $blksize$ contiguous iterations. We use an auxiliary array $la(1:numproc, 1:arysize)$, which is initially set to zero, to keep the latest iteration, all the way, that access the array element for each

processor. Namely, $la(p,j)$ records the latest iteration getting access to the array element j for processor p (la is the abbreviation of *latest access*). The algorithm of our parallel inspector is shown in Fig. 2, which consists of two phases:

1. Parallel recording phase (lines 3 to 10).

Step 1. If $la(p,w(i))$ or $la(p,r(i))$ is not zero, let them be c and d respectively, then record $pw(i)$ as c and $pr(i)$ as d , meaning that c is the predecessor iteration of iteration i for accessing the array element $w(i)$ and d is the predecessor iteration of iteration i for accessing the array element $r(i)$. The arrays pw and pr are initially set to zero.

Step 2. Set $la(p,w(i))$ and $la(p,r(i))$ to current iteration i . This means that, for processor p , iteration i is the latest iteration that access the array elements $w(i)$ and $r(i)$.

2. Parallel patching phase (lines 11 to 24). For each iteration i , if $pw(i)=0$ then find the largest-numbered processor q , where $q <$ the current processor p , such that $la(q,w(i))$ is not zero. Assume now that $la(q,w(i))=j$, then set $pw(i)=j$. This means that j is the real predecessor iteration of iteration i in accessing the array element $w(i)$. In the same way, if $pr(i)=0$ we find the largest-numbered processor $t <$ the current processor p such that $la(t,r(i))=k \neq 0$, and set $pr(i)=k$ to mean that, in reality, k is the predecessor iteration of iteration i in accessing the array element $r(i)$.

For the example in previous subsection, we assume that there are two processors. By block distribution, iterations 1 to 6 are charged by processor 1 and iterations 7 to 12 by processor 2. The contents of the auxiliary array la and the predecessor iteration table pw and pr for processor 1 and processor 2 are shown in Fig. 3(a) and Fig. 3(b), respectively, when the parallel recording phase is in progress. Remember that processor 1 is in charge of iterations 1 to 6 and processor 2 in

charge of iterations 7 to 12. For easier reference, the access pattern in Section 3.2 is also included in Fig. 3. We can see that, when processor1 is dealing with iteration 4 it will write array element 1 (since $w(4)=1$) and read array element 3 (since $r(4)=3$). But array element 1 has been accessed (both read and written in this example) by iteration 3 (this can be seen from $la(1,1)=3$), and array element 3 has been accessed (written in this example) by iteration 1 (since $la(1,3)=1$). Therefore, the predecessor iterations of iteration 4 are iteration 3 for writing array element 1, and iteration 1 for reading array element 3. The predecessor iteration table is hence recorded as $pw(4)=3$ and $pr(4)=1$. This behavior is encoded in lines 5 and 6 of our inspector algorithm in Fig. 2. After this, $la(1,1)$ will be updated from 3 to 4 to mean that iteration 4 is now the latest iteration of accessing the array element 1, and $la(1,3)$ will be updated from 1 to 4 to mean that iteration 4 is now the latest iteration of accessing the array element 3. This behavior is coded in lines 7 and 8 in Fig. 2. Since the values in array la after iteration i is processed might be changed when iteration $i+1$ is dealt with, for clear demonstration, the new updated values are **bold-faced** in Fig. 3(a).

In the parallel patching phase, let us see, in Fig. 3(b), why $pw(7)$ remains 0 (unchanged) and how $pr(7)$ is changed to 2 by processor 2. Since $pw(7)$ in Fig. 3(a) is zero, this means no prior iteration in processor 2 accesses the same array element with iteration 7. But from $w(7)=8$, we know that iteration 7 *writes* array element 8. Hence, we have to trace back to processor 1 to check whether it has accessed this element. By checking $la(1,8)=0$, we find that no iteration in processor 1 has ever accessed array element 8. Therefore, $pw(7)$ remains unchanged. In the same argument, $pr(7)$ in Fig. 3(a) is zero, which means no prior iteration in processor 2 accesses the same array element with iteration 7. But $r(7)=4$ indicates that iteration 7 *reads* array element 4. By tracing back to processor 1, we find $la(1,4)=2$. This means that, in processor 1, iteration 2 also accesses array element 4. Hence, the predecessor iteration of iteration 7 should be 2 for

read access and *pr*(7) is changed to 2. The behavior is coded in lines 13 to 20 and lines 21 to 28 for *write* access and *read* access respectively. The predecessor iterations that have been changed are also **bold-faced** in Fig. 3(b) for illustration. As for the time complexity, it is easy to see that our parallel inspector takes $O(\text{numiter})$ time.

3.4 The Parallel Scheduler

The algorithm of our parallel scheduler is presented in Fig. 4. Scheduling the loop iterations into wavefronts becomes very easy once the predecessor iteration table is available. In the beginning, the wavefront table *wf*(1:*numiter*) is set to zero to indicate that all loop iterations have not been scheduled.

Like our parallel inspector, the loop iterations are distributed into processors in block fashion. For each wavefront number, all processors simultaneously examine the iterations they are charged in series (lines 8 and 9) to see if they can be assigned into the current wavefront. Only the iterations that have not been scheduled yet (line 10) and whose predecessor iterations, for both *write* and *read* access, have been scheduled can be arranged into the current wavefront (lines 11 and 12). This procedure repeats until all loop iterations have been scheduled (line 5). The maximum wavefront number will be referred as *cpl* (critical path length) because it is the length of critical path in the directed acyclic graph representing the cross-iteration dependency in the target loop.

4 Experimental Results

We performed our experiments on ALR Quad6, a shared-memory multiprocessor machine with four Pentium Pro 200MHz processors and 128MB global memory. The synthetic loop in Fig. 1(a) was added with run-time procedures and OpenMP parallelization directives, and then compiled using the pgf77 compiler.

We intended to evaluate the impact of parallelism degree on the performance of two run-time schemes: PRS (Predecessor_Reference_Scheme) by Rauchwerger *et al.*[8], and PIS (Predecessor_Iteration_Scheme) by us. Let the grain size (workload) be 200us, the array size be 2048, and the iteration number vary from 2048 to 65536. The access patterns are generated using a probabilistic method. Since the accessed area (array size) is fixed to 2048, a loop with 65536 iterations will be more serial in comparison to a loop with 2048 iterations. Table 1 shows the execution time measured in each run-time phase.

Fig. 5 is a speedup comparison for two run-time schemes. We can see that PIS always obtains a higher speedup and still has a satisfactory speedup of 2.8 in the worst case. The overhead comparison in Fig. 6 shows that the processing overhead ((inspector time + scheduler time) / sequential loop time) of PRS is dramatically larger than that of PIS. This is because the scheduler of PRS examines all the references repeatedly as mentioned in Section 3.1.

5 Conclusion

In the parallelization of partially parallel loops, the biggest challenge is to find a parallel execution schedule that can fully extract the potential parallelism but incurs run-time overhead as little as possible. In this paper, we present a new run-time parallelization scheme PIS, which use the information of predecessor/successor iterations instead of predecessor references to eliminate the processing overhead of repeatedly examining all the references in PRS. The predecessor iteration table can be constructed in parallel with no synchronization and our parallel scheduler can quickly produce the wavefront schedule with its help. From either theoretical time analysis or experimental results, our run-time scheme reveals better speedup and less processing overhead than PRS.

References

- [1] Chen, D. K., Yew, P. C., Torrellas, J.: An Efficient Algorithm for the Run-Time Parallelization of Doacross Loops. Proc. 1994 Supercomputing (1994) 518-527
- [2] Huang, T. C., Hsu, P. H.: The SPNT Test: A New Technology for Run-Time Speculative Parallelization of Loops. *Lecture Notes in Computer Science* Vol. 1366. Springer-Verlag, Berlin Heidelberg New York (1998) 177-191
- [3] Huang, T. C., Hsu, P. H., Sheng, T. N.: Efficient Run-Time Scheduling for Parallelizing Partially Parallel Loops," *J. Information Science and Engineering* 14(1), (1998) 255-264
- [4] Leung, S. T., Zahorjan, J.: Improving the Performance of Run-Time Parallelization. Proc. 4th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming, (1993) 83-91
- [5] Leung, S. T., Zahorjan, J.: Extending the Applicability and Improving the Performance of Run-Time Parallelization. Tech. Rep. 95-01-08, Dept. CSE, Univ. of Washington (1995)
- [6] Midkiff, S., Padua, D.: Compiler Algorithms for Synchronization. *IEEE Trans. Comput.* C-36, 12, (1987) 1485-1495
- [7] Polychronopoulos, C.: Compiler Optimizations for Enhancing Parallelism and Their Impact on Architecture Design. *IEEE Trans. Comput.* C-37, 8, (1988) 991-1004
- [8] Rauchwerger, L., Amato, N., Padua, D.: A Scalable Method for Run-Time Loop Parallelization. *Int. J. Parallel Processing*, 26(6), (1995) 537-576
- [9] Rauchwerger, L., Padua, D.: The LRPD Test: Speculative Run-Time Parallelization of Loops with Privatization and Reduction Parallelization. *IEEE Trans. Parallel and Distributed Systems*, 10(2), (1999) 160-180
- [10] Saltz, J., Mirchandaney, R., Crowley, K.: Run-time Parallelization and Scheduling of Loops. *IEEE Trans. Comput.* 40(5), (1991) 603-612
- [11] Xu, C., Chaudhary, V.: Time-Stamping Algorithms for Parallelization of Loops at Run-Time. Proc. 11th Int. Parallel Processing Symp. (1997)
- [12] Zhu, C. Q., Yew, P. C.: A Scheme to Enforce Data Dependence on Large Multiprocessor Systems. *IEEE Trans. Software Eng.* 13(6), (1987) 726-739

```

do i = 1, numiter
    x(w(i)) = ...
    y(i) = x(r(i)) ...      w(1:12)=[3 4 1 1 5 2 8 1 8 5 7 2]
    ...                    r(1:12)= [5 6 1 3 7 2 4 3 8 7 8 1]

end do

```

(a) An irregular loop

(b) Synthetic access pattern

iter. pred. iter.	1	2	3	4	5	6	7	8	9	10	11	12
pw(i)	0	0	0	3	1	0	0	4	7	5	10	6
pr(i)	0	0	0	1	0	0	2	4	7	5	9	8

(c) Predecessor iteration table

iter. wavefront	1	2	3	4	5	6	7	8	9	10	11	12
wf(i)	1	1	1	2	2	1	2	3	3	3	4	4

(d) Wavefront schedule

Figure 1. An example to demonstrate the operations of our scheme.

```

/* The construction of predecessor iteration table */
1 pr(1:numiter)=0
2 pw(1:numiter)=0
/* Parallel recording phase */
3 doall p=1,numproc
4   do i=(p-1)*(numiter/numproc)+1,p*(numiter/numproc)
5     if (la(p,w(i)).ne.0) then pw(i)=la(p,w(i))
6     if (la(p,r(i)).ne.0) then pr(i)=la(p,r(i))
7     la(p,w(i))=i
8     la(p,r(i))=i
9   enddo
10 enddoall
/* Parallel patching phase */
11 doall p=2,numproc
12   do i=(p-1)*(numiter/numproc)+1,p*(numiter/numproc)
13     if(pw(i).eq.0) then
14       do j=p-1,1,-1
15         if(la(j,w(i)).ne.0) then
16           pw(i)=la(j,w(i))
17           goto S1
18         endif
19       enddo
20 s1: endif
21     if(pr(i).eq.0) then
22       do j=p-1,1,-1
23         if(la(j,r(i)).ne.0) then
24           pr(i)=la(j,r(i))

```

```

25         goto S2
26     endif
27 enddo
28 S2: endif
29 enddo
30 enddoall

```

Figure 2. The algorithm of inspector.

Processor 1

iter. access. element	1	2	3	4	5	6
W(i)	3	4	1	1	5	2
R(i)	5	6	1	3	7	2
la(1,1)	0	0	3	4	4	4
la(1,2)	0	0	0	0	0	6
la(1,3)	1	1	1	4	4	4
la(1,4)	0	2	2	2	2	2
la(1,5)	1	1	1	1	5	5
la(1,6)	0	2	2	2	2	2
la(1,7)	0	0	0	0	5	5
la(1,8)	0	0	0	0	0	0

Processor 2

iter. access. element	7	8	9	10	11	12
w(i)	8	1	8	5	7	2
r(i)	4	3	8	7	8	1
la(2,1)	0	8	8	8	8	12
la(2,2)	0	0	0	0	0	12
la(2,3)	0	8	8	8	8	8
la(2,4)	7	7	7	7	7	7
la(2,5)	0	0	0	10	10	10
la(2,6)	0	0	0	0	0	0
la(2,7)	0	0	0	10	11	11
la(2,8)	7	7	9	9	11	11

iter. pred. iter.	1	2	3	4	5	6	7	8	9	10	11	12
pw(i)	0	0	0	3	1	0	0	0	7	0	10	0
pr(i)	0	0	0	1	0	0	0	0	7	0	9	8

(a)

iter. pred. iter.	1	2	3	4	5	6	7	8	9	10	11	12
pw(i)	0	0	0	3	1	0	0	4	7	5	10	6
pr(i)	0	0	0	1	0	0	2	4	7	5	9	8

(b)

Figure 3. (a) The contents of array *la* and the predecessor iteration table for processor 1 and processor 2 when the parallel recording phase is in progress. (b) The predecessor iteration table after parallel patching phase is finished.


```

/* Schedule iterations into wavefronts in parallel */
1  wf(1:numiter)=0
2  wf(0)=1
3  done=.false.
4  wfnum=0
/* Repeated until all iterations are scheduled */
5  do while (done.eq..false.)
6    done=.true.
7    wfnum=wfnum+1
8    doall p=1,numproc
9      do i=(p-1)*(numiter/numproc)+1,p*(numiter/numproc)
10       if (wf(i).eq.0) then
11         if (wf(pw(i)).ne.0.and.wf(pr(i)).ne.0) then
12           wf(i)=wfnum
13         else
14           done=.false.
15         endif
16       endif
17     enddo
18   enddoall
19 enddo

```

Figure 4. The algorithm of the scheduler.

Table 1. Execution time measured on two run-time schemes.

Number of Iterations		2048	4096	8192	16384	32768	65536
Critical Path Length		9	16	30	52	105	199
Sequential Loop Time		413	828	1657	3322	6651	13322
Inspector Time (msec)	PRS	164	274	326	332	390	484
	PIS	3	6	10	31	48	90
Scheduler Time (msec)	PRS	4	14	59	314	2283	8470
	PIS	2	5	11	47	117	460
Executor Time (msec)	PRS	117	229	469	945	1914	4010
	PIS	114	231	477	980	1975	4210
Total Execution Time	PRS	285	517	854	1591	4587	12964
	PIS	119	242	498	1058	2140	4760
Speedup	PRS	1.45	1.60	1.94	2.09	1.45	1.03
	PIS	3.47	3.42	3.33	3.14	3.11	2.80

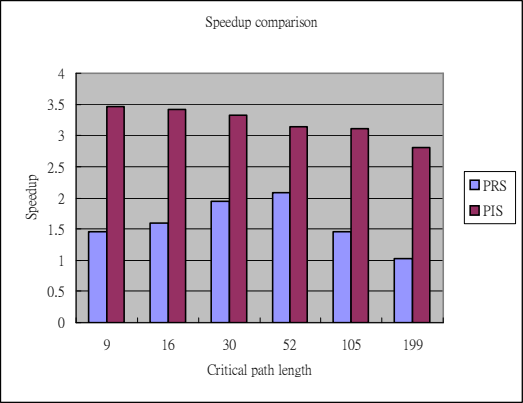


Figure 5. Speedup comparison.

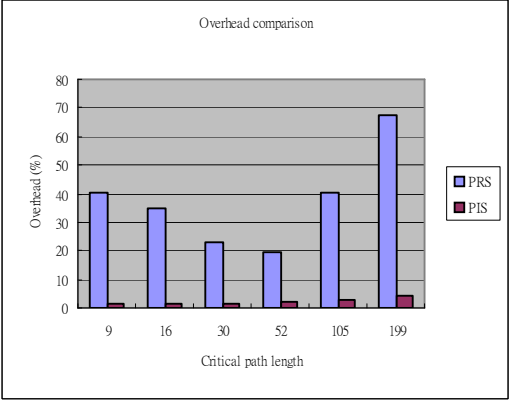


Figure 6. Overhead comparison.